LAPORAN

PRAKTIKUM BASIS DATA

Oleh :

1. Cornelia Nasywa Puji Agustin (264311065)

2. Muhammad Ahnaf Ghifari (264311075)

3. Neisya Grazia Aulia Firdianty (264311079)

JURUSAN TEKNIK
PROGRAM STUDI TEKNOLOGI REKAYASA PERANGKAT LUNAK
POLITEKNIK NEGERI MADIUN

2026

10.3 TUGAS/UNJUK KERJA (PRAKTIKUM & PRESENTASI)

1. **Eksplorasi DBMS dan Arsitektur:** Identifikasi DBMS yang digunakan, versi, storage engine, perangkat lunak pengelola, dan gambarkan arsitektur implementasi sistem.

EKSPLORASI DBMS DAN ARSITEKTUR

Identifikasi MySQL dan DBeaver

1. Identifikasi DBMS yang Digunakan

DBMS (Database Management System) yang digunakan dalam praktikum ini adalah MySQL. MySQL berfungsi untuk mengelola penyimpanan data, membuat tabel, mengatur relasi antartabel, serta menjalankan perintah SQL. Untuk mengelola database, digunakan aplikasi DBeaver. DBeaver menyediakan antarmuka yang membantu pengguna terhubung ke MySQL, menjalankan query SQL, serta melihat dan mengelola isi tabel.

Komponen	Keterangan
DBMS	MySQL
Aplikasi pengelola	DBeaver
Server lokal	Laragon (jika digunakan)
Bahasa query	SQL
Database	Isi dengan nama database proyek
Storage Engine	InnoDB (pastikan melalui hasil pemeriksaan)
Versi DBMS	Isi berdasarkan hasil SELECT VERSION()

2. Aplikasi Pengelola Basis Data

Aplikasi yang digunakan untuk mengelola database adalah DBeaver. DBeaver merupakan aplikasi database client yang dapat digunakan untuk terhubung ke berbagai DBMS, termasuk MySQL dan MariaDB.

Kelebihan DBeaver

1. Memiliki tampilan antarmuka yang memudahkan pengguna mengelola database.
2. Mendukung penulisan dan eksekusi perintah SQL.
3. Dapat digunakan untuk melihat struktur tabel dan relasi antartabel.
4. Mendukung berbagai jenis DBMS, termasuk MySQL dan MariaDB.
5. Menyediakan fitur untuk mengimpor dan mengekspor data.

Keterbatasan DBeaver

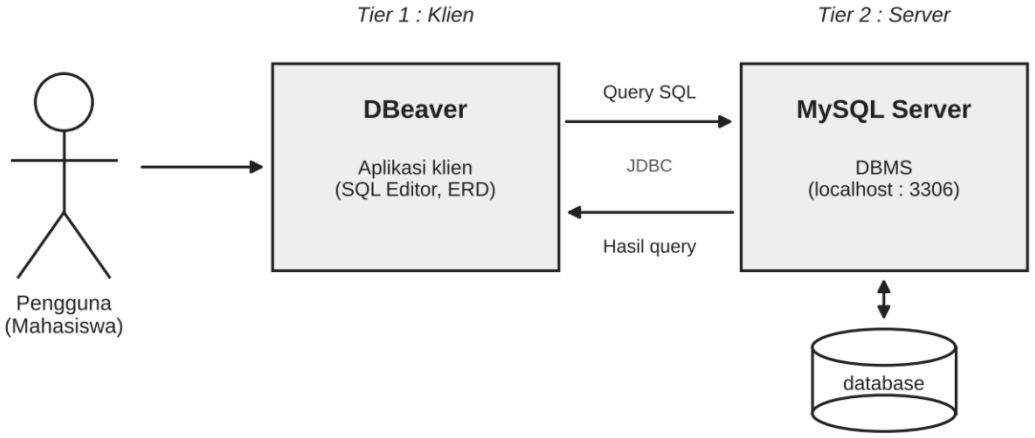
1. Membutuhkan koneksi ke database server agar dapat mengelola data.
2. Pengguna pemula perlu mempelajari beberapa fitur dan pengaturannya.
3. Beberapa fitur lanjutan hanya tersedia pada edisi berbayar.
4. Lebih berat dan memakai lebih banyak RAM dibanding phpMyAdmin
5. Hanya berperan sebagai klien, sehingga server DBMS (misalnya melalui XAMPP) tetap harus dijalankan terpisah.

Alasan Pemilihan DBeaver

DBeaver dipilih karena memiliki fitur yang cukup lengkap untuk kebutuhan praktikum dan pengerjaan proyek PjBL. Aplikasi ini memudahkan proses menjalankan perintah SQL, membuat tabel, mengatur relasi, serta melihat data dalam database. Selain itu, DBeaver dapat digunakan bersama MySQL sehingga sesuai dengan kebutuhan pengelolaan basis data proyek.

3.Arsitektur Implementasi Sistem

Dalam praktikum ini, DBeaver berperan sebagai aplikasi klien yang berkomunikasi secara langsung dengan MySQL Server. Oleh karena itu, proses pengelolaan database melalui DBeaver menggunakan arsitektur Two-Tier Architecture.



Penjelasan Gambar 1.

Pengguna. Pengguna adalah mahasiswa yang mengelola basis data. Pengguna tidak berhubungan langsung dengan MySQL, tetapi bekerja melalui DBeaver, misalnya dengan menulis query SQL, membuka tabel, atau membuat ERD.

Tier 1 (Klien): DBeaver. Tier pertama adalah lapisan yang berinteraksi dengan pengguna. DBeaver menyediakan antarmuka untuk menulis perintah SQL dan menampilkan hasilnya. DBeaver tidak menyimpan maupun mengolah data sendiri; tugasnya adalah mengirim permintaan ke server dan menampilkan jawaban yang diterima.

Komunikasi antartier. DBeaver mengirim perintah SQL (query SQL) ke MySQL Server menggunakan driver JDBC. Server kemudian membalas dengan hasil query berupa data atau pesan hasil eksekusi. Komunikasi ini berlangsung dua arah dan terjadi secara langsung tanpa melalui perantara.

Tier 2 (Server): MySQL Server. Tier kedua adalah DBMS yang menerima dan memproses query. Server membaca atau menulis data pada database melalui storage engine InnoDB, sehingga hubungan antara server dan database berlangsung dua arah. Alamat localhost:3306 menunjukkan bahwa server berjalan di komputer yang sama dan menerima koneksi melalui port 3306.

Alasan termasuk Two-Tier. Arsitektur ini hanya terdiri atas dua tingkat, yaitu klien dan server, yang saling berkomunikasi secara langsung. Tidak terdapat application layer berupa web server atau aplikasi perantara yang memproses logika bisnis. Hal inilah yang membedakannya dari Three-Tier Architecture.

Komponen	Implementasi	Peran
Pengguna	Mahasiswa	Menulis dan menjalankan perintah SQL.
Aplikasi klien	DBeaver	Antarmuka pengelolaan dan pengiriman query ke server.
Server DBMS	MySQL Server (localhost:3306)	Memproses query dan mengelola basis data.
Storage engine	InnoDB	Menyimpan data dan menjaga integritas transaksi.
Database	db_praktikum	Menyimpan tabel dan relasi proyek.

4. Alasan Pemilihan DBeaver untuk Proyek PjBL

1. **Memiliki tampilan yang mudah dipahami dan digunakan.**
   DBeaver memiliki antarmuka (*user interface*) yang bersih, rapi, dan intuitif. Hal ini memudahkan anggota kelompok dalam menjelajahi struktur database, melihat tabel, kolom, serta isi data secara visual tanpa harus terus-menerus mengetikkan perintah teks.
2. **Mendukung koneksi dengan DBMS MySQL.**

   DBeaver kompatibel dan dapat dihubungkan dengan berbagai macam Database Management System (DBMS), termasuk **MySQL** yang banyak digunakan dalam proyek PjBL mahasiswa (biasanya dijalankan melalui XAMPP atau Laragon).

3. **Memudahkan pembuatan dan pengelolaan tabel.**

   Aplikasi ini menyediakan fitur *Graphical User Interface* (GUI) yang praktis untuk membuat tabel baru, menambah atau mengubah tipe data kolom, hingga menghapus tabel dengan cepat sesuai dengan perancangan basis data sistem yang sedang dibuat.

4. **Mendukung pengaturan relasi antar tabel menggunakan Primary Key dan Foreign Key.**

   DBeaver sangat membantu dalam menyusun dan memvisualisasikan hubungan (*relationship*) antar tabel, seperti relasi *One-to-Many* atau *Many-to-Many*, sehingga integritas data pada proyek dapat terstruktur dengan baik.

5. **Menyediakan editor SQL untuk menjalankan query.**

   Bagi mahasiswa yang ingin menguji perintah SQL secara manual, DBeaver menyediakan *SQL Editor* lengkap dengan fitur penyorotan sintaks (*syntax highlighting*) untuk menulis perintah seperti SELECT, INSERT, UPDATE, DELETE, dan JOIN.

6. **Mendukung operasi CRUD (Create, Read, Update, Delete).**

   Melalui DBeaver, pengembang dapat dengan mudah melakukan manipulasi data dasar secara langsung, mulai dari menambah data baru (*Create*), melihat data (*Read*), memperbarui isi data (*Update*), hingga menghapus data (*Delete*).

7. **Memudahkan pemeriksaan struktur dan isi database.**

   Proses *debugging* atau pengecekan kesalahan pada database menjadi lebih mudah karena kita bisa langsung melihat jenis data, kunci utama (*Primary Key*), kunci tamu (*Foreign Key*), serta isi record dari tabel-tabel proyek secara real-time.

8. **Mendukung fitur import dan export data.**

   DBeaver dilengkapi fitur untuk memasukkan data dari file eksternal (seperti CSV atau SQL dump) ke dalam database (*import*) maupun mencadangkan/menyimpan data dari database ke format file lain (*export*).

9. **Mendukung proses pengujian dan pengelolaan basis data proyek PjBL.**

   Secara keseluruhan, DBeaver menjadi perangkat yang diandalkan kelompok untuk menguji logika basis data, memastikan seluruh tabel saling terhubung dengan benar, dan menjamin data siap digunakan oleh aplikasi utama proyek.

10. **Memiliki versi gratis yang cukup untuk kebutuhan praktikum dan proyek mahasiswa.**

    DBeaver menyediakan edisi **Community (Open Source)** yang dapat diunduh dan digunakan secara gratis tanpa batasan waktu. Versi gratis ini sudah sangat memadai untuk mengakomodasi seluruh kebutuhan dasar pembuatan database dalam skala proyek mahasiswa tanpa harus mengeluarkan biaya lisensi.

Keterbatasan DBeaver, seperti pemakaian memori yang lebih besar dan proses backup yang memerlukan utilitas bawaan MySQL, tidak menghambat kebutuhan proyek pada tahap perancangan dan pengelolaan basis data. Dengan pertimbangan tersebut, DBeaver dinilai sebagai pilihan yang tepat.

2. **Implementasi dan Integritas Basis Data:** Buat minimal tiga tabel yang saling berelasi, gunakan PK dan FK, kemudian praktikkan operasi CRUD serta pengujian integritas referensial.

TABEL YANG SALING BERELASI

1. TABEL PENGGUNA

```
INSERT INTO pengguna
(id_pengguna, nama, no_hp, email, kata_sandi, peran, tanggal_daftar)
VALUES
(1,  'Budi Santoso',   '081234567890', 'budi@example.test',   'hash_budi',   'petani', '2026-01-01 08:00:00'),
(2,  'Siti Aminah',    '081234567891', 'siti@example.test',   'hash_siti',   'petani', '2026-01-02 09:00:00'),
(3,  'Admin',          '081234567892', 'admin@example.test',  'hash_admin',  'admin',  '2026-01-03 10:00:00'),
(4,  'Yanto',          '086767676891', 'yanto@example.test',  'hash_yanto',  'petani', '2026-02-03 12:00:00'),
(5,  'Slamet Riyadi',  '082133445501', 'slamet@example.test', 'hash_slamet', 'petani', '2026-02-10 07:30:00'),
(6,  'Sri Wahyuni',    '085233445502', 'sri@example.test',    'hash_sri',    'petani', '2026-02-18 13:15:00'),
(7,  'Agus Prasetyo',  '081333445503', 'agus@example.test',   'hash_agus',   'petani', '2026-03-05 09:45:00'),
(8,  'Wulan Sari',     '087833445504', 'wulan@example.test',  'hash_wulan',  'petani', '2026-03-21 16:20:00'),
(9,  'Joko Susilo',    '081533445505', 'joko@example.test',   'hash_joko',   'petani', '2026-04-02 08:10:00'),
(10, 'Dewi Lestari',   '085733445506', 'dewi@example.test',   'hash_dewi',   'petani', '2026-04-14 11:00:00');
```

Properties | Data | Diagram

Show SQL | Enter a SQL expression to filter results (use Ctrl+Space)

Grid | Text

	id_wilayah	kode_wilayah	provinsi	kabupaten	kecamatan	desa
1	1	JTM-01	Jawa Timur	Madiun	Dolopo	Dolopo
2	2	JTM-02	Jawa Timur	Madiun	Mejayan	Mejayan
3	3	JTM-03	Jawa Timur	Madiun	Wungu	Wungu
4	4	JTM-04	Jawa Timur	Madiun	Geger	Geger
5	5	JTM-05	Jawa Timur	Madiun	Saradan	Saradan
6	6	JTM-06	Jawa Timur	Kota Madiun	Kartoharjo	Kartoharjo
7	7	JTM-07	Jawa Timur	Magetan	Maospati	Maospati
8	8	JTM-08	Jawa Timur	Magetan	Plaosan	Plaosan
9	9	JTM-09	Jawa Timur	Ngawi	Paron	Paron
10	10	JTM-10	Jawa Timur	Ponorogo	Jetis	Jetis

2. TABEL WILAYAH

Properties | Data | Diagram

Show SQL | Enter a SQL expression to filter results (use Ctrl+Space)

Grid | Text

	id_wilayah	kode_wilayah	provinsi	kabupaten	kecamatan	desa
1	1	JTM-01	Jawa Timur	Madiun	Dolopo	Dolopo
2	2	JTM-02	Jawa Timur	Madiun	Mejayan	Mejayan
3	3	JTM-03	Jawa Timur	Madiun	Wungu	Wungu
4	4	JTM-04	Jawa Timur	Madiun	Geger	Geger
5	5	JTM-05	Jawa Timur	Madiun	Saradan	Saradan
6	6	JTM-06	Jawa Timur	Kota Madiun	Kartoharjo	Kartoharjo
7	7	JTM-07	Jawa Timur	Magetan	Maospati	Maospati
8	8	JTM-08	Jawa Timur	Magetan	Plaosan	Plaosan
9	9	JTM-09	Jawa Timur	Ngawi	Paron	Paron
10	10	JTM-10	Jawa Timur	Ponorogo	Jetis	Jetis

3. TABEL TANAMAN

```
INSERT INTO tanaman
(id_tanaman, nama_tanaman, masa_tanam_hari, curah_hujan_min, curah_hujan_maks, suhu_min, suhu_maks)
VALUES
(1, 'Padi',         120, 100, 300, 20, 35),
(2, 'Jagung',       100,  50, 200, 18, 32),
(3, 'Kedelai',       85,  60, 200, 20, 33),
(4, 'Cabai Rawit',  110,  60, 160, 18, 32),
(5, 'Bawang Merah',  65,  30, 120, 20, 32),
(6, 'Tebu',         360, 100, 250, 22, 34);
```

Properties | Data | Diagram

Show SQL | Enter a SQL expression to filter results (use Ctrl+Space)

	id_t	nama_tanaman	masa_tanam_hari	curah_hujan_min	curah_hujan_maks	suhu_min	suhu_maks
1	1	Padi	120	100	300	20	35
2	2	Jagung	100	50	200	18	32
3	3	Kedelai	85	60	200	20	33
4	4	Cabai Rawit	110	60	160	18	32
5	5	Bawang Merah	65	30	120	20	32
6	6	Tebu	360	100	250	22	34

4. TABEL PERINGATAN CUACA

```
INSERT INTO peringatan_cuaca
(id_peringatan, id_wilayah, jenis_peringatan, tingkat, deskripsi, waktu_mulai, waktu_berakhir)
VALUES
(1, 1, 'Hujan Lebat',   'Siaga',   'Potensi hujan lebat disertai petir di wilayah Dolopo dan sekitarnya.',    '2026-11-09 13:00:00', '2026-11-09 19:00:00'),
(2, 2, 'Angin Kencang', 'Waspada', 'Angin kencang berpotensi merusak tanaman muda di sekitar Mejayan.',       '2026-11-10 10:00:00', '2026-11-10 17:00:00'),
(3, 5, 'Banjir',        'Awas',    'Risiko genangan di lahan rendah sekitar Saradan akibat hujan berkepanjangan.', '2026-11-12 06:00:00', '2026-11-13 18:00:00'),
(4, 7, 'Hujan Lebat',   'Waspada', 'Hujan intensitas tinggi di Maospati pada sore hingga malam hari.',        '2026-11-11 15:00:00', '2026-11-11 22:00:00'),
(5, 9, 'Genangan Air',  'Waspada', 'Potensi genangan pada lahan sawah di sekitar Paron.',                     '2026-11-13 08:00:00', '2026-11-14 12:00:00');
```

Properties | Data | Diagram

Show SQL | Enter a SQL expression to filter results (use Ctrl+Space)

	id_peringatan	id_wilayah	jenis_peringatan	tingkat	deskripsi	waktu_mulai	waktu_berakhir
1	1	1	Hujan Lebat	Siaga	Potensi hujan lebat disertai petir di wilayah	2026-11-09 13:00:00	2026-11-09 19:00:00
2	2	2	Angin Kencang	Waspada	Angin kencang berpotensi merusak tanama	2026-11-10 10:00:00	2026-11-10 17:00:00
3	3	5	Banjir	Awas	Risiko genangan di lahan rendah sekitar Sar	2026-11-12 06:00:00	2026-11-13 18:00:00
4	4	7	Hujan Lebat	Waspada	Hujan intensitas tinggi di Maospati pada so	2026-11-11 15:00:00	2026-11-11 22:00:00
5	5	9	Genangan Air	Waspada	Potensi genangan pada lahan sawah di seki	2026-11-13 08:00:00	2026-11-14 12:00:00

5. TABEL CUACA

```
INSERT INTO cuaca
(id_cuaca, id_wilayah, waktu, jenis_data, suhu, kelembapan, curah_hujan, kecepatan_angin, kondisi)
VALUES
(1,  1, '2026-11-08 07:00:00', 'observasi', 26.0, 82,  2.0,  6.0, 'berawan'),
(2,  1, '2026-11-08 14:00:00', 'observasi', 31.5, 65,  0.0,  8.0, 'cerah berawan'),
(3,  2, '2026-11-08 09:00:00', 'observasi', 28.0, 75,  5.5,  9.5, 'hujan ringan'),
(4,  5, '2026-11-08 08:00:00', 'observasi', 27.0, 85, 14.0,  7.0, 'hujan sedang'),
(5,  6, '2026-11-08 12:00:00', 'observasi', 32.0, 60,  0.0,  5.0, 'cerah'),
(6,  7, '2026-11-08 15:00:00', 'observasi', 29.0, 78,  8.0, 12.0, 'hujan ringan'),
(7,  1, '2026-11-09 15:00:00', 'prakiraan', 25.0, 90, 40.0, 18.0, 'hujan lebat'),
(8,  2, '2026-11-10 12:00:00', 'prakiraan', 29.0, 70,  3.0, 25.0, 'berangin'),
(9,  5, '2026-11-12 09:00:00', 'prakiraan', 25.5, 92, 55.0, 14.0, 'hujan lebat'),
(10, 9, '2026-11-13 10:00:00', 'prakiraan', 26.0, 88, 30.0, 10.0, 'hujan sedang');
```

*<localhost> Script-15 | cuaca | lahan | notifikasi | pengguna | peringatan_cuaca | rekomendasi | tanaman | wilayah

Properties | Data | Diagram

Show SQL | Enter a SQL expression to filter results (use Ctrl+Space)

	id_cuaca	id_wilayah	waktu	jenis_data	suhu	kelembapan	curah_hujan	kecepatan_angin	kondisi
1	1	1	2026-11-08 07:00:00	observasi	26	82	2	6	berawan
2	2	1	2026-11-08 14:00:00	observasi	31.5	65	0	8	cerah berawan
3	3	2	2026-11-08 09:00:00	observasi	28	75	5.5	9.5	hujan ringan
4	4	5	2026-11-08 08:00:00	observasi	27	85	14	7	hujan sedang
5	5	6	2026-11-08 12:00:00	observasi	32	60	0	5	cerah
6	6	7	2026-11-08 15:00:00	observasi	29	78	8	12	hujan ringan
7	7	1	2026-11-09 15:00:00	prakiraan	25	90	40	18	hujan lebat
8	8	2	2026-11-10 12:00:00	prakiraan	29	70	3	25	berangin
9	9	5	2026-11-12 09:00:00	prakiraan	25.5	92	55	14	hujan lebat
10	10	9	2026-11-13 10:00:00	prakiraan	26	88	30	10	hujan sedang

6. TABEL NOTIFIKASI

```
INSERT INTO notifikasi
(id_notifikasi, id_pengguna, id_peringatan, isi_pesan, waktu_kirim, status_baca)
VALUES
(1, 1,  1, 'Peringatan hujan lebat untuk wilayah Dolopo, Kabupaten Madiun.',    '2026-11-08 18:00:00', FALSE),
(2, 2,  2, 'Peringatan angin kencang untuk wilayah Mejayan.',                   '2026-11-09 17:00:00', FALSE),
(3, 6,  3, 'Peringatan banjir tingkat Awas untuk wilayah Saradan.',             '2026-11-11 18:00:00', FALSE),
(4, 7,  4, 'Peringatan hujan lebat untuk wilayah Maospati, Magetan.',           '2026-11-10 20:00:00', TRUE),
(5, 9,  5, 'Peringatan genangan air untuk wilayah Paron, Ngawi.',               '2026-11-12 16:00:00', FALSE),
(6, 1,  3, 'Info banjir Saradan, pantau kondisi lahan Anda di wilayah terdekat.', '2026-11-11 18:30:00', TRUE);
```

Properties | Data | Diagram

Show SQL | Enter a SQL expression to filter results (use Ctrl+Space)

Grid | Text

	id_notifikasi	id_pengguna	id_peringatan	isi_pesan	waktu_kirim	status_baca
1	1	1	1	Peringatan hujan lebat untuk wilayah Dolo	2026-11-08 18:00:00	0
2	2	2	2	Peringatan angin kencang untuk wilayah M	2026-11-09 17:00:00	0
3	3	6	3	Peringatan banjir tingkat Awas untuk wilay	2026-11-11 18:00:00	0
4	4	7	4	Peringatan hujan lebat untuk wilayah Maos	2026-11-10 20:00:00	1
5	5	9	5	Peringatan genangan air untuk wilayah Par	2026-11-12 16:00:00	0
6	6	1	3	Info banjir Saradan, pantau kondisi lahan A	2026-11-11 18:30:00	1

7. TABEL LAHAN

```
-- 4. Tabel LAHAN
CREATE TABLE lahan (
    id_lahan INT PRIMARY KEY AUTO_INCREMENT,
    id_pengguna INT NOT NULL,
    id_wilayah INT NOT NULL,
    nama_lahan VARCHAR(100),
    latitude DECIMAL(10,8),
    longitude DECIMAL(11,8),
    luas_m2 DECIMAL(15,2),
    CONSTRAINT fk_lahan_pengguna
        FOREIGN KEY (id_pengguna) REFERENCES pengguna(id_pengguna)
        ON UPDATE CASCADE ON DELETE CASCADE,
    CONSTRAINT fk_lahan_wilayah
        FOREIGN KEY (id_wilayah) REFERENCES wilayah(id_wilayah)
        ON UPDATE CASCADE ON DELETE CASCADE
) ENGINE=InnoDB;

-- 5. Tabel PERINGATAN_CUACA
CREATE TABLE peringatan_cuaca (
    id_peringatan INT PRIMARY KEY AUTO_INCREMENT,
    id_wilayah INT NOT NULL,
    jenis_peringatan VARCHAR(100),
    tingkat VARCHAR(50),
    deskripsi TEXT,
    waktu_mulai DATETIME,
    waktu_berakhir DATETIME,
    CONSTRAINT fk_peringatan_wilayah
        FOREIGN KEY (id_wilayah) REFERENCES wilayah(id_wilayah)
        ON UPDATE CASCADE ON DELETE CASCADE
) ENGINE=InnoDB;
```

Properties | Data | Diagram

Show SQL | Enter a SQL expression to filter results (use Ctrl+Space)

	id_lahan	id_pengguna	id_wilayah	nama_lahan	latitude	longitude	luas_m2
1	1	1	1	Sawah Budi Dolopo	-7.5921	111.6048	2,400
2	2	2	2	Sawah Siti Mejayan	-7.5532	111.6561	1,800
3	3	4	3	Ladang Yanto Wungu	-7.6641	111.6182	1,500
4	4	5	4	Lahan Slamet Geger	-7.7063	111.6334	3,000
5	5	6	5	Kebun Sri Saradan	-7.5291	111.7572	5,000
6	6	7	7	Sawah Agus Maospati	-7.6312	111.4391	2,000
7	7	8	8	Ladang Wulan Plaosan	-7.6694	111.3287	900
8	8	9	9	Sawah Joko Paron	-7.4582	111.3345	2,600
9	9	10	10	Lahan Dewi Jetis	-7.8423	111.5124	1,200
10	10	1	6	Kebun Budi Kartoharjo	-7.6284	111.5237	600

8. TABEL REKOMENDASI

```
INSERT INTO rekomendasi
(id_rekomendasi, id_lahan, id_tanaman, jenis, tanggal_awal, tanggal_akhir, alasan, dibuat_pada)
VALUES
(1, 1, 1, 'tanam', '2026-11-15', '2026-12-15', 'Memasuki musim hujan, curah hujan dan suhu sesuai untuk padi.',          '2026-11-08 08:00:00'),
(2, 2, 2, 'tunda', '2026-11-10', '2026-11-17', 'Angin kencang diperkirakan, tunda tanam jagung sampai kondisi stabil.',  '2026-11-08 08:30:00'),
(3, 3, 3, 'tanam', '2026-11-12', '2026-12-05', 'Curah hujan sedang, cocok untuk penanaman kedelai.',                    '2026-11-08 09:00:00'),
(4, 4, 2, 'tanam', '2026-11-14', '2026-12-10', 'Tanah Geger mendukung jagung, suhu dan curah hujan masih dalam rentang.', '2026-11-08 09:15:00'),
(5, 5, 6, 'tunda', '2026-11-12', '2026-11-20', 'Risiko banjir di Saradan, tunda penanaman tebu.',                       '2026-11-08 09:30:00'),
(6, 6, 1, 'tanam', '2026-11-18', '2026-12-18', 'Hujan lebat sementara, tanam padi setelah genangan surut.',            '2026-11-08 10:00:00'),
(7, 7, 4, 'tunda', '2026-11-11', '2026-11-18', 'Hujan lebat berpotensi merusak bibit cabai, tunda penanaman.',         '2026-11-08 10:15:00'),
(8, 8, 1, 'tunda', '2026-11-13', '2026-11-22', 'Potensi genangan di Paron, tunggu air surut sebelum tanam padi.',      '2026-11-08 10:30:00'),
(9, 9, 5, 'tanam', '2026-11-16', '2026-11-30', 'Lahan Jetis relatif kering, cocok untuk bawang merah.',                '2026-11-08 10:45:00');
```

Show SQL | Enter a SQL expression to filter results (use Ctrl+Space)

	id_rek	id_lahan	id_tanaman	jenis	tanggal_awal	tanggal_akhir	alasan	dibuat_pada
1	1	1	1	tanam	2026-11-15	2026-12-15	Memasuki musim hujan, curah hujan dan s	2026-11-08 08:00:00
2	2	2	2	tunda	2026-11-10	2026-11-17	Angin kencang diperkirakan, tunda tanam j	2026-11-08 08:30:00
3	3	3	3	tanam	2026-11-12	2026-12-05	Curah hujan sedang, cocok untuk penanan	2026-11-08 09:00:00
4	4	4	2	tanam	2026-11-14	2026-12-10	Tanah Geger mendukung jagung, suhu dar	2026-11-08 09:15:00
5	5	5	6	tunda	2026-11-12	2026-11-20	Risiko banjir di Saradan, tunda penanaman	2026-11-08 09:30:00
6	6	6	1	tanam	2026-11-18	2026-12-18	Hujan lebat sementara, tanam padi setelah	2026-11-08 10:00:00
7	7	7	4	tunda	2026-11-11	2026-11-18	Hujan lebat berpotensi merusak bibit cabai	2026-11-08 10:15:00
8	8	8	1	tunda	2026-11-13	2026-11-22	Potensi genangan di Paron, tunggu air suru	2026-11-08 10:30:00
9	9	9	5	tanam	2026-11-16	2026-11-30	Lahan Jetis relatif kering, cocok untuk bawa	2026-11-08 10:45:00

9. Memeriksa Relasi Tabel

```
SHOW TABLES;
SHOW CREATE TABLE lahan;
SHOW CREATE TABLE rekomendasi;
```

Results 1

SHOW TABLES | Enter a SQL expression to filter resu

	Tables_in_sdg_rkpl+basdat_db
1	cuaca
2	lahan
3	notifikasi
4	pengguna
5	peringatan_cuaca
6	rekomendasi
7	tanaman
8	wilayah

Results 1

SHOW CREATE TABLE lahan | Enter a SQL expression to filter results (use Ctrl+Space)

	Table	Create Table
1	lahan	CREATE TABLE `lahan` (¶ `id_lahan` int NOT NULL AUTO_INCREMENT,¶ `id_pengguna` int DEFAULT NULL,¶ `id_wilayah` int D

SHOW CREATE TABLE rekomendasi | Enter a SQL expression to filter results (use Ctrl+Space)

	Table	Create Table
1	rekomendasi	CREATE TABLE `rekomendasi` (¶ `id_rekomendasi` int NOT NULL AUTO_INCREMENT,¶ `id_lahan` int DEFAULT NULL,¶ `id_tan

- **Primary key**
  Kolom unik yang menjadi pengenal utama untuk setiap baris data di dalam sebuah tabel dan tidak boleh bernilai kosong (NULL).
  Contoh di proyek :
  - Pada table pengguna, *Primary Key*-nya adalah **id_pengguna**.
  - Pada tabel wilayah, *Primary Key*-nya adalah **id_wilayah**.
  - Pada tabel cuaca, *Primary Key*-nya adalah **id_cuaca**
  - Pada tabel tanaman, *Primary Key*-nya adalah **id_tanaman**
  - Pada tabel rekomendasi, *Primary Key*-nya adalah **id_rekomendasi**
  - Pada tabel lahan, *Primary Key*-nya adalah **id_lahan**
  - Pada tabel notifikasi, *Primary Key*-nya adalah **id_notifikasi**
  - Pada tabel peringatan cuaca, *Primary Key*-nya adalah **id_peringatan**
  - Berdasarkan struktur database sistem ini secara umum, tabel lain juga memiliki Primary Key masing-masing, seperti id_pengguna pada tabel pengguna dan id_wilayah pada tabel wilayah).

- **Foreign key**
  Kolom pada suatu tabel yang nilainya merujuk ke *Primary Key* pada tabel lain, yang berfungsi untuk menghubungkan antar tabel dan menjaga integritas referensial data.
  - **Tabel Peringatan Cuaca:** Kolom id_wilayah merujuk ke tabel Wilayah.
  - **Tabel Cuaca:** Kolom id_wilayah merujuk ke tabel Wilayah.
  - **Tabel Notifikasi:** Kolom id_pengguna (merujuk ke tabel Pengguna) dan kolom id_peringatan (merujuk ke tabel Peringatan Cuaca).
  - **Tabel Rekomendasi:** Kolom id_lahan dan kolom id_tanaman (merujuk ke tabel Tanaman).
  - **Tabel Lahan** juga memiliki Foreign Key id_pengguna dan id_wilayah).

- **Relasi one-to-many**
  Hubungan antar tabel di mana satu baris data pada tabel induk (*parent*) dapat berelasi dengan banyak baris data pada tabel anak (*child*).
  - **PENGGUNA ke LAHAN:** Satu pengguna dapat memiliki **banyak** lahan.
  - **WILAYAH ke LAHAN:** Satu wilayah mencakup **banyak** lahan.
  - **Wilayah ke Peringatan Cuaca:** Satu wilayah (id_wilayah = 1) dapat memiliki banyak data peringatan cuaca.
  - **Wilayah ke Cuaca:** Satu wilayah dapat memiliki banyak catatan data cuaca (baik observasi maupun prakiraan) dari waktu ke waktu.
  - **Pengguna ke Notifikasi:** Satu pengguna dapat menerima banyak pesan notifikasi sistem.
  - **Peringatan Cuaca ke Notifikasi:** Satu peringatan cuaca dapat memicu pengiriman banyak notifikasi ke beberapa pengguna terkait.
  - **Tanaman ke Rekomendasi:** Satu jenis tanaman (misalnya Padi dengan id_tanaman = 1) dapat direkomendasikan ke banyak data lahan melalui tabel Rekomendasi.

- **Relasi antarentitas**
  Hubungan logis dan konseptual antar tabel yang mendefinisikan bagaimana sistem informasi mengolah data secara terintegrasi:
  1. Relasi antara PENGGUNA dan LAHAN
  Seorang **Pengguna** (petani) dapat memiliki **banyak Lahan** pertanian di berbagai lokasi. Sebaliknya, satu data lahan spesifik hanya dimiliki oleh satu pengguna terdaftar.
  2. **Relasi antara WILAYAH dengan LAHAN, PERINGATAN_CUACA, dan CUACA**
  - Suatu **Wilayah** (kecamatan/desa) menjadi acuan lokasi tempat **banyak Lahan** berada.
  - Suatu Wilayah dapat memiliki **banyak catatan Peringatan Cuaca** (seperti peringatan hujan lebat atau angin kencang).
  - Suatu Wilayah juga menjadi cakupan tempat terkumpulnya **banyak data Cuaca** (baik data observasi maupun prakiraan harian).

  3. Relasi antara LAHAN, TANAMAN, dan REKOMENDASI
  Bentuk Hubungan: Tabel REKOMENDASI bertindak sebagai *junction table* (tabel perantara) yang memecah relasi *Many-to-Many* antara **Lahan** dan **Tanaman**.
  Satu **Lahan** dapat menerima **banyak** data **Rekomendasi** jenis tanaman dari waktu ke waktu, dan satu jenis **Tanaman** dapat direkomendasikan ke **banyak Lahan** yang berbeda karakteristiknya.

  4. Relasi antara PENGGUNA, PERINGATAN_CUACA, dan NOTIFIKASI
  - Satu **Pengguna** dapat menerima **banyak Notifikasi** dari sistem.
  - Satu kejadian **Peringatan Cuaca** di suatu wilayah dapat memicu pengiriman **banyak Notifikasi** kepada para pengguna yang terdampak.

- **Constraint yang diterapkan**
  Aturan atau batasan yang diterapkan pada kolom tabel untuk menjamin kevalidan, keunikan, dan konsistensi data yang dimasukkan ke database.
  - **PRIMARY KEY**: Memastikan setiap data pada kolom ID bersifat unik dan wajib ada.
  - **FOREIGN KEY**: Mencegah adanya data yatim (*orphan data*) dengan memastikan nilai yang dimasukkan pada kolom relasi harus sudah terdaftar di tabel induknya.
  - **Tipe Data (int, varchar/string, decimal, datetime, date, boolean):** Batasan format nilai isi kolom. Contohnya kolom suhu dan kelembapan menggunakan tipe angka desimal (decimal), kolom status_baca pada notifikasi menggunakan tipe logika (boolean / 0 dan 1), serta kolom waktu menggunakan datetime

PRAKTIK OPERASI CRUD

1. CREATE – Menambahkan Data

```
-- CRUD
-- ======================================================
-- CREATE
INSERT INTO pengguna (nama, no_hp, email, kata_sandi, peran)
VALUES ('Mundi', '081200000000', 'mundi@example.test', 'hash_mundi', 'petani');
```

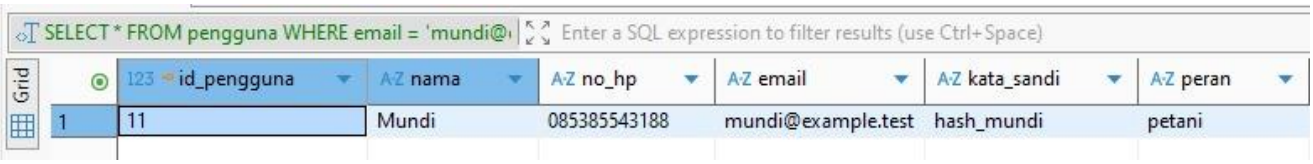
2. **READ – Membaca Data**

```
-- READ
SELECT id_pengguna, nama, email, peran FROM pengguna;
SELECT * FROM lahan WHERE luas_m2 > 500;
```

3. **UPDATE – Memperbarui Data**

```
UPDATE pengguna
SET no_hp = '085385543188'
WHERE email = 'mundi@example.test';

SELECT * FROM pengguna WHERE email = 'mundi@example.test';
```



SELECT * FROM pengguna WHERE email = 'mundi@ | Enter a SQL expression to filter results (use Ctrl+Space)

Grid

	id_pengguna	nama	no_hp	email	kata_sandi	peran
1	11	Mundi	085385543188	mundi@example.test	hash_mundi	petani

4. **DELETE – Menghapus Data**

```
DELETE FROM pengguna
WHERE email = 'mundi@example.test';

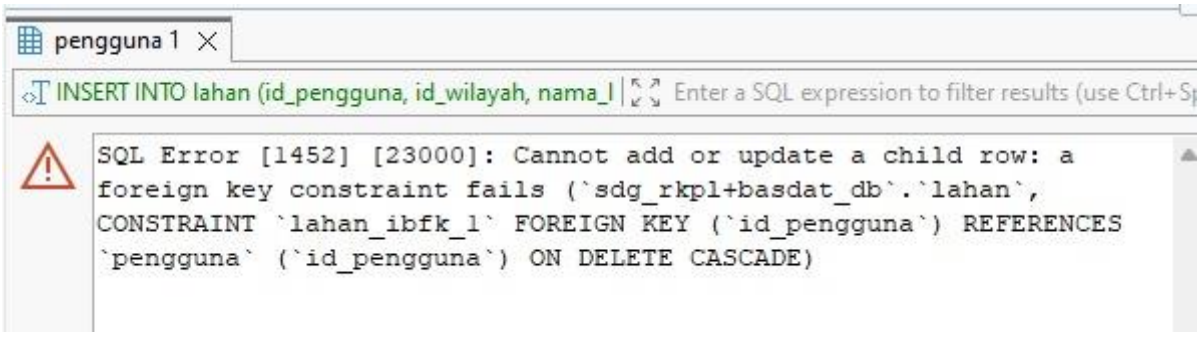
SELECT * FROM pengguna;
```



10	10	Dewi Lestari	085733445506	dewi@example.test	hash_dewi	petani	2

5. **Menguji Integritas Referensial**

```
-- A. Insert lahan dengan id_pengguna yang tidak ada -> HARUS GAGAL
INSERT INTO lahan (id_pengguna, id_wilayah, nama_lahan)
VALUES (999, 1, 'Uji FK gagal');
```



pengguna 1

INSERT INTO lahan (id_pengguna, id_wilayah, nama_l | Enter a SQL expression to filter results (use Ctrl+Sp

```
SQL Error [1452] [23000]: Cannot add or update a child row: a
foreign key constraint fails (`sdg_rkpl+basdat_db`.`lahan`,
CONSTRAINT `lahan_ibfk_1` FOREIGN KEY (`id_pengguna`) REFERENCES
`pengguna` (`id_pengguna`) ON DELETE CASCADE)
```

3. **Analisis Performa dan Transaksi:** Jalankan query, buat indeks yang relevan, gunakan EXPLAIN untuk mengamati rencana eksekusi, dan demonstrasikan penggunaan transaksi COMMIT dan ROLLBACK.

1. . Pengujian COMMIT

```sql
START TRANSACTION;

INSERT INTO pengguna
(id_pengguna, nama, email, kata_sandi, peran)
VALUES
(20, 'Uji Commit', 'commit@example.test', 'hash_commit', 'petani');

INSERT INTO lahan
(id_lahan, id_pengguna, id_wilayah, nama_lahan, luas_m2)
VALUES
(200, 20, 1, 'Lahan Commit', 500);

INSERT INTO rekomendasi
(id_rekomendasi, id_lahan, id_tanaman, jenis, tanggal_awal, tanggal_akhir, alasan)
VALUES
(2000, 200, 1, 'tanam', '2026-11-01', '2026-12-01', 'Uji commit');

COMMIT;

SELECT * FROM pengguna WHERE id_pengguna = 20;
SELECT * FROM lahan WHERE id_lahan = 200;
SELECT * FROM rekomendasi WHERE id_rekomendasi = 2000;
```

Name	Value
Updated Rows	1
Execute time	0.006s
Start time	Sun Nov 08 16:31:55 WIB 2026
Finish time	Sun Nov 08 16:31:55 WIB 2026
Query	INSERT INTO pengguna
	(id_pengguna, nama, email, kata_sandi, peran)
	VALUES
	(20, 'Uji Commit', 'commit@example.test', 'hash_commit', 'petani')

SELECT * FROM pengguna WHERE id_pengguna = 20 | Enter a SQL expression to filter results (use Ctrl+Space)

Grid

	id_pengguna	nama	no_hp	email	kata_sandi	per
1	20	Uji Commit	[NULL]	commit@example.test	hash_commit	petani

Updated Rows	1
Execute time	0.004s
Start time	Sun Nov 08 16:32:37 WIB 2026
Finish time	Sun Nov 08 16:32:37 WIB 2026
Query	INSERT INTO rekomendasi
	(id_rekomendasi, id_lahan, id_tanaman, jenis, tanggal_awal, tanggal_akhir, alasan)
	VALUES
	(2000, 200, 1, 'tanam', '2026-11-01', '2026-12-01', 'Uji commit')

rekomendasi 1

SELECT * FROM rekomendasi WHERE id_rekomendas | Enter a SQL expression to filter results (use Ctrl+Space)

Grid

	id_rekomendasi	id_lahan	id_tanaman	jenis	tanggal_awal	tanggal_akhir	alasan	dibuat_pada
1	2,000	200	1	tanam	2026-11-01	2026-12-01	Uji commit	[NULL]

Statistics 1

Name	Value
Updated Rows	1
Execute time	0.009s
Start time	Sun Nov 08 16:32:16 WIB 2026
Finish time	Sun Nov 08 16:32:16 WIB 2026
Query	INSERT INTO lahan
	(id_lahan, id_pengguna, id_wilayah, nama_lahan, luas_m2)
	VALUES
	(200, 20, 1, 'Lahan Commit', 500)

```
SELECT * FROM lahan WHERE id_lahan = 200
```

Enter a SQL expression to filter results (use Ctrl+Space)

Grid

	id_lahan	id_pengguna	id_wilayah	nama_lahan	latitude	longitude	luas_m2
1	200	20	1	Lahan Commit	[NULL]	[NULL]	500

2. Pengujian ROLLBACK

```
START TRANSACTION;
DELETE FROM pengguna WHERE id_pengguna = 1;
SELECT * FROM lahan WHERE id_pengguna = 1;
SELECT * FROM rekomendasi WHERE id_lahan = 1;
ROLLBACK;
```

Name	Value
Updated Rows	0
Execute time	1.782s
Start time	Sun Nov 08 16:24:00 WIB 2026
Finish time	Sun Nov 08 16:24:02 WIB 2026
Query	START TRANSACTION

```
SELECT * FROM lahan WHERE id_pengguna = 1
```

Enter a SQL expression to filter results (use Ctrl+Space)

Grid

Text

Record

id_lahan	id_pengguna	id_wilayah	nama_lahan	latitude	longitude	luas_m2

Activate Wir

Name	Value
Updated Rows	1
Execute time	0.01s
Start time	Sun Nov 08 16:24:37 WIB 2026
Finish time	Sun Nov 08 16:24:37 WIB 2026
Query	DELETE FROM pengguna WHERE id_pengguna = 1

SELECT * FROM rekomendasi WHERE id_lahan = 1 | Enter a SQL expression to filter results (use Ctrl+Space)

id_rekomendasi	id_lahan	id_tanaman	jenis	tanggal_awal	tanggal_akhir	alasan	dibuat_pada

Name	Value
Updated Rows	0
Execute time	0.02s
Start time	Sun Nov 08 16:26:14 WIB 2026
Finish time	Sun Nov 08 16:26:14 WIB 2026
Query	ROLLBACK

BEFORE ROLLBACK

lahan 1

SELECT * FROM lahan WHERE id_lahan = 201 | Enter a SQL expression to filter results (use Ctrl+Space)

	id_lahan	id_pengguna	id_wilayah	nama_lahan	latitude	longitude	luas_m2
1	201	21	1	Lahan Rollback	[NULL]	[NULL]	500

AFTER ROLLBACK

lahan 1

SELECT * FROM lahan WHERE id_lahan = 201 | Enter a SQL expression to filter results (use Ctrl+Space)

id_lahan	id_pengguna	id_wilayah	nama_lahan	latitude	longitude	luas_m2

BEFORE ROLLBACK

SELECT * FROM pengguna WHERE id_pengguna = 21 | Enter a SQL expression to filter results (use Ctrl+Space)

	id_pengguna	nama	no_hp	email	kata_sandi	peran	tanggal_daftar
1	21	Uji Rollback	[NULL]	rollback@example.test	hash_rollback	petani	[NULL]

AFTER ROLLBACK

SELECT * FROM pengguna WHERE id_pengguna = 21 | Enter a SQL expression to filter results (use Ctrl+Space)

id_pengguna	nama	no_hp	email	kata_sandi	peran	tanggal_daftar

UJI ROLLBACK

```sql
START TRANSACTION;

INSERT INTO pengguna
(id_pengguna, nama, email, kata_sandi, peran)
VALUES
(21, 'Uji Rollback', 'rollback@example.test', 'hash_rollback', 'petani');

INSERT INTO lahan
(id_lahan, id_pengguna, id_wilayah, nama_lahan, luas_m2)
VALUES
(201, 21, 1, 'Lahan Rollback', 500);

ROLLBACK;

SELECT * FROM pengguna WHERE id_pengguna = 21;
SELECT * FROM lahan WHERE id_lahan = 201;
```

Analisis Hasil Transaksi

1. Apa perbedaan COMMIT dan ROLLBACK?

COMMIT : Perintah dalam transaksi database yang berfungsi untuk **menyimpan secara permanen** semua perubahan data (seperti INSERT, UPDATE, DELETE) yang telah dilakukan sejak perintah START TRANSACTION dijalankan. Setelah perintah COMMIT dieksekusi, perubahan tersebut tidak dapat dibatalkan dan sudah tersimpan secara permanen di dalam database. Sedangkan,

ROLLBACK: Perintah yang berfungsi untuk **membatalkan seluruh perubahan** yang terjadi selama transaksi berlangsung sejak START TRANSACTION dan mengembalikan kondisi database seperti semula sebelum transaksi dimulai. Perintah ini biasanya digunakan jika terjadi kesalahan atau kegagalan pada salah satu proses.

2. Mengapa operasi transaksi perlu dilakukan secara atomik?

Operasi transaksi perlu dilakukan secara atomik (mengikuti prinsip **ACID** — *Atomicity*) agar seluruh rangkaian perintah di dalam satu blok transaksi diperlakukan sebagai satu kesatuan tunggal. Artinya, seluruh perintah harus berhasil dieksekusi bersama-sama, atau jika ada satu saja perintah yang gagal, semuanya harus dibatalkan (ROLLBACK).

Hal ini mencegah terjadinya inkonsistensi data atau data yang setengah-setengah tersimpan (misalnya data pengguna berhasil tersimpan, tetapi data lahan dan rekomendasinya gagal, sehingga database tetap bersih dan valid).

3. Apa risiko jika transaksi penjualan hanya berhasil menyimpan tabel pesanan tetapi gagal menyimpan detailnya?

Jika sistem hanya berhasil menyimpan data di tabel pesanan (master) tetapi gagal menyimpan data detail pesanan, maka risiko yang akan terjadi meliputi:

- **Inkonsistensi Data (Data Tidak Valid):** Data pesanan tercatat masuk ke dalam sistem, tetapi produk atau rincian barang yang dibeli oleh pelanggan tidak tercatat sama sekali.
- **Kerugian Finansial dan Operasional:** Toko atau sistem tidak mengetahui barang apa saja yang harus dikirimkan atau diproses, yang berujung pada komplain pelanggan atau selisih stok barang.
- **Kerusakan Integritas Relasi:** Hubungan (*relationship*) antara tabel pesanan dan tabel detail pesanan menjadi terputus karena data pendukungnya tidak ada.

4. Mengapa penggunaan storage engine InnoDB penting pada praktikum ini?

Penggunaan storage engine InnoDB sangat penting karena InnoDB merupakan salah satu storage engine bawaan MySQL yang mendukung penuh fitur transaksi (ACID), termasuk mekanisme START TRANSACTION, COMMIT, dan ROLLBACK. Jika menggunakan storage engine yang tidak mendukung transaksi (seperti MyISAM), perintah seperti ROLLBACK tidak akan berfungsi dan perubahan data akan langsung tersimpan secara permanen meskipun terjadi kesalahan di tengah jalan. Oleh karena itu, InnoDB wajib digunakan agar pengujian kontrol transaksi ini dapat berjalan dengan benar.

PENGUJIAN QUERY SEBELUM INDEX

```
EXPLAIN
SELECT id_cuaca, id_wilayah, waktu, suhu, kondisi
FROM cuaca
WHERE waktu >= '2025-01-01 00:00:00'
  AND waktu <  '2025-02-01 00:00:00';
```

HASIL

Results 1

EXPLAIN SELECT id_cuaca, id_wilayah, waktu, suhu, k | Enter a SQL expression to filter results (use Ctrl+Space)

	id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	1	SIMPLE	cuaca	[NULL]	ALL	[NULL]	[NULL]	[NULL]	[NULL]	10	11.11	Using where

BUAT INDEX

```
CREATE INDEX idx_cuaca_waktu
ON cuaca(waktu);

CREATE INDEX idx_cuaca_wilayah_waktu
ON cuaca(id_wilayah, waktu);

SHOW INDEX FROM cuaca;
```

HASIL

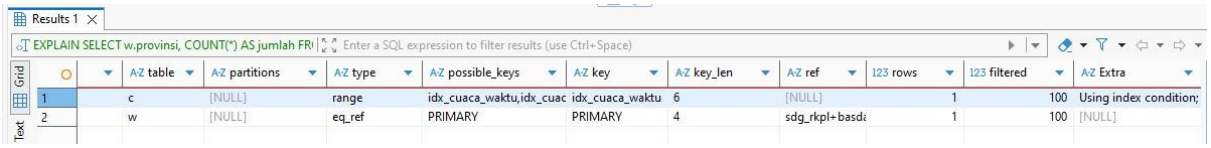
	Table	Non_unique	Key_name	Seq_in_index	Column_name	Collation	Cardinality	Sub_part	Packed	Nu
1	cuaca	0	PRIMARY	1	id_cuaca	A	10	[NULL]	[NULL]	
2	cuaca	1	idx_cuaca_waktu	1	waktu	A	10	[NULL]	[NULL]	YES
3	cuaca	1	idx_cuaca_wilayah_waktu	1	id_wilayah	A	6	[NULL]	[NULL]	YES
4	cuaca	1	idx_cuaca_wilayah_waktu	2	waktu	A	10	[NULL]	[NULL]	YES

EXPLAIN SESUDAH INDEX

```
EXPLAIN
SELECT id_cuaca, id_wilayah, waktu, suhu, kondisi
FROM cuaca
WHERE waktu >= '2025-01-01 00:00:00'
  AND waktu <  '2025-02-01 00:00:00';

EXPLAIN
SELECT w.provinsi, COUNT(*) AS jumlah
FROM cuaca c
JOIN wilayah w ON c.id_wilayah = w.id_wilayah
WHERE c.waktu >= '2025-01-01 00:00:00'
  AND c.waktu <  '2025-02-01 00:00:00'
GROUP BY w.provinsi;
```

HASIL



Analisis Penggunaan Index

1. Apa perbedaan query sebelum dan sesudah penambahan indeks?

   Sebelum menggunakan indeks: Database melakukan **Full Table Scan** (pencarian penuh), yaitu memeriksa setiap baris data dari awal sampai akhir tabel satu per satu untuk menemukan data yang cocok. Hal ini membuat waktu eksekusi (execution time) menjadi jauh lebih lambat, terutama pada tabel dengan jumlah data yang sangat besar.

   Sesudah menggunakan indeks: Database menggunakan struktur data khusus (seperti *B-Tree*) layaknya daftar pustaka atau indeks buku. Database dapat langsung melompat ke lokasi data yang dituju tanpa harus membaca seluruh baris tabel, sehingga proses pencarian (*filtering* atau *joining*) menjadi **jauh lebih cepat dan efisien**.

2. Apa fungsi kolom key pada hasil EXPLAIN?

   Kolom key pada hasil perintah EXPLAIN berfungsi untuk **menunjukkan indeks mana yang benar-benar dipilih dan digunakan oleh *database optimizer*** saat mengeksekusi sebuah query.

   - Jika kolom key bernilai NULL, artinya database tidak menggunakan indeks apa pun dan terpaksa melakukan *Full Table Scan*.
   - Jika kolom key terisi nama indeks (misalnya idx_email), artinya query tersebut berhasil memanfaatkan indeks yang telah dibuat untuk mempercepat pencarian data.

3. Apakah indeks selalu digunakan oleh optimizer?

Tidak selalu. *Database optimizer* (mesin pengoptimal query pada DBMS) tidak otomatis menggunakan indeks untuk setiap query. Optimizer akan menganalisis biaya (*cost-based optimization*) dan memutuskan tidak menggunakan indeks jika:

- **Jumlah data terlalu sedikit:** Jika tabel hanya berisi sedikit baris, melakukan *Full Table Scan* seringkali lebih cepat daripada harus membaca struktur indeks terlebih dahulu.
- **Selektivitas rendah:** Jika data yang dicari mencakup sebagian besar dari isi tabel (misalnya mencari data jenis kelamin yang 90% bernilai sama), penggunaan indeks menjadi tidak efisien.
- **Kolom di dalam fungsi/operasi matematika:** Jika kolom yang memiliki indeks dimanipulasi dengan fungsi (contoh: WHERE YEAR(tanggal) = 2026), indeks biasanya gagal dibaca oleh optimizer.

4. Mengapa indeks yang terlalu banyak dapat mengurangi performa operasi INSERT, UPDATE, dan DELETE?

   Operasi INSERT, UPDATE, dan DELETE melibatkan perubahan data (penambahan, pembaruan, atau penghapusan). Alasan performanya menurun jika terlalu banyak indeks adalah:

   - **Beban Pembaruan Struktur Indeks:** Setiap kali baris data baru ditambahkan atau diubah, database tidak hanya memperbarui tabel utamanya, tetapi **wajib memperbarui seluruh struktur indeks** yang terikat pada tabel tersebut agar tetap terurut dan sinkron.
   - **Overhead I/O Disk:** Semakin banyak indeks, semakin banyak pula file atau halaman indeks di penyimpanan yang harus ditulis ulang oleh sistem, yang berujung pada lambatnya waktu proses tulis (*write performance*).

5. Pada situasi apa indeks perlu ditambahkan?

   Kolom yang sering menjadi kriteria pencarian (WHERE): Kolom yang sering dipanggil untuk menyaring data, seperti email, username, atau status_pesanan.

   Kolom relasi (*Primary Key* dan *Foreign Key*): Kolom yang digunakan untuk menghubungkan antar tabel (JOIN), karena operasi penggabungan tabel sangat bergantung pada kecepatan pencarian kunci relasi.

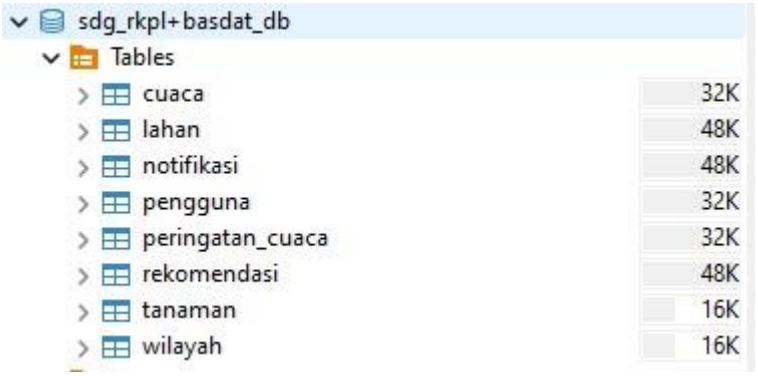
   Kolom yang sering diurutkan (ORDER BY) atau dikelompokkan (GROUP BY): Menggunakan indeks dapat menghindari proses pengurutan data secara manual oleh memori database yang memakan waktu lama.

   Tabel berukuran besar: Pada tabel yang memiliki ribuan hingga jutaan baris data, penambahan indeks memberikan dampak peningkatan kecepatan yang paling signifikan.

Pemeliharaan Basis Data (Backup & Restore)

- **BACK UP DAN RESTORE**

BEFORE



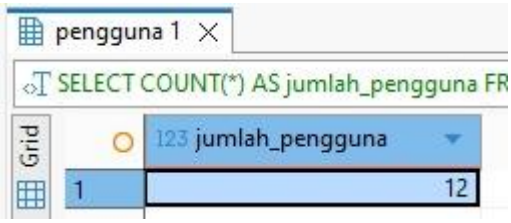
AFTER DI RESTORE



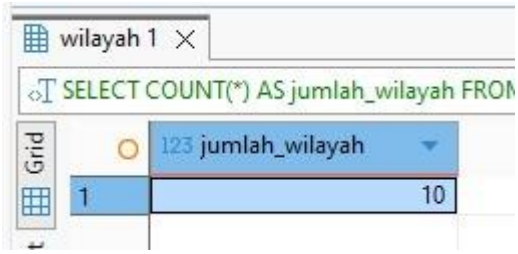
VALIDASI HASIL RESTORE

```
SELECT COUNT(*) AS jumlah_pengguna FROM pengguna;
SELECT COUNT(*) AS jumlah_wilayah FROM wilayah;
SELECT COUNT(*) AS jumlah_cuaca FROM cuaca;
```

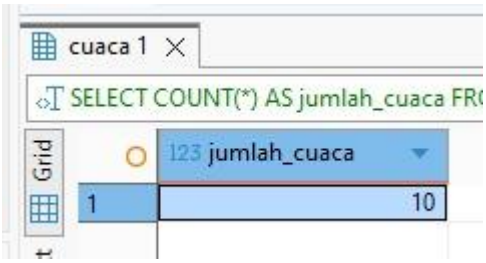
MENAMPILKAN JUMLAH PENGGUNA



MENAMPILKAN JUMLAH WILAYAH



MENAMPILKAN JUMLAH CUACA



4. **Pemeliharaan, Presentasi, dan Git:** Lakukan ekspor dan impor basis data, dokumentasikan hasil praktik, simpan script SQL ke Git, dan demonstrasikan hasil implementasi kepada dosen.

